Session 08 — Tuples

Python for Everyone · Lecture notes

> **How to read the code boxes:** grey boxes with a yellow bar contain Python code. The box labelled **Output** next to it (or below it) shows what Python prints when you run that code. In a Jupyter notebook, if the last line of a cell is just a value (like `l` below), the notebook displays it; this is also shown as output.

Tuples

- A **tuple** is much like a list, with two key differences.
- **Syntax:** a tuple is defined with round brackets `()` instead of square brackets `[]`. Indexing and slicing work the same way.

```
l = [1, 2, 3, 4, 5]
t = (1, 2, 3, 4, 5)
print(t[0], l[0])
print(t[2:4], l[2:4])
```

Output:

```
1 1
(3, 4) [3, 4]
```

- **Behavior:** lists are **mutable**: you can change an entry or add new ones.

```
l[1] = 'hello'
l.append('one more thing')
l
```

Output:

```
[1, 'hello', 3, 4, 5, 'one more thing']
```

- Tuples are **immutable**: you cannot change their entries or add new ones.

```
t[0] = 'why?'
```

Output:

```
TypeError: 'tuple' object does not support item assignment
```

- Tuples have no `.append()`: `t.append('hm?')` gives an `AttributeError`.

Why do we need tuples?

- Use a tuple for **small, fixed-length** values that **belong together**: the x and y coordinates of a point, a name and an age, the parameters of a measurement.

```
point = (3, 8)
```

- A list would also work, but a point always has two coordinates, while the length of a list can change. Much of the functionality of a list is not needed, or even unwanted.

- Lists are mutable, so a list **cannot** be a key in a dictionary. Tuples are immutable, so they **can**:

```
D = {}
D[(1, 2)] = "works!"
D
```

Output:

```
{(1, 2): 'works!'}
```

```
D[[1, 2]] = "oh, no!"
```

Output:

```
TypeError: unhashable type: 'list'
```

Flat vs nested data

- The same data can be structured in different ways. Both have advantages and disadvantages. For example, flight durations between pairs of cities:
  - **flat:** one dictionary, with a tuple of two cities as the key;
  - **nested:** a dictionary of dictionaries, first city then second city.

```
flights_flat = {
    ("New York", "London"): 7.0,
    ("London", "Paris"): 1.2,
    ("New York", "Tokyo"): 14.0
}
flights_nested = {
    "New York": {"London": 7.0, "Tokyo": 14.0},
    "London": {"Paris": 1.2}
}
print(flights_flat[("London", "Paris")])
print(flights_nested["London"]["Paris"])
```

Output:

```
1.2
1.2
```

- Finding the longest flight in the nested version needs a loop inside a loop:

```
max_duration = -1
for data in flights_nested.values():
    for duration in data.values():
        if max_duration < duration:
            max_duration = duration
max_duration
```

Output:

```
14.0
```

Implicit tuples

- Strictly speaking, you do not need `()` to define a tuple. Tuples are often created this way for convenience:

```
t = 1, 2, 3
t
```

Output:

```
(1, 2, 3)
```

- We already did this to display two variables on the last line of a notebook cell. Note the `()` around the output: `x, y` creates a tuple, and the notebook displays that tuple.

```
x = 10
y = 2*x
x, y
```

Output:

```
(10, 20)
```

Unpacking

- **Unpacking** assigns the elements of a tuple to several variables at once:

```
x, y = (10, 20)
print(x)
print(y)
```

Output:

```
10
20
```

- It is commonly used when a function **returns multiple values**. The function returns a tuple, which can be unpacked into separate variables:

```
def min_and_max(L):
    return min(L), max(L)

L = [12, 34, 42, -18, 137]
t = min_and_max(L)
print(t, type(t))
minima, maxima = min_and_max(L)
print(minima)
```

Output:

```
(-18, 137) <class 'tuple'>
-18
```

Where did we see tuples before?

- `.items()` of a dictionary gives the key and the value as a tuple. The loop `for key, value in d.items():` unpacks it.

```
penguin_weights = {"Emperor": 30.0, "King": 13.5, "Adelie": 4.5}
for t in penguin_weights.items():
    print(t, type(t))
```

Output:

```
('Emperor', 30.0) <class 'tuple'>
('King', 13.5) <class 'tuple'>
('Adelie', 4.5) <class 'tuple'>
```

Iterating over two lists: `zip`

- To calculate the body-mass index, BMI = weight / height², we need the weight and the height of the same person. A loop over `heights` alone is not enough.
- One way is to iterate over all possible indices with `range(len(...))`:

```
heights = [1.65, 1.72, 1.80]  # in meters
weights = [62, 75, 82]        # in kilograms
for i in range(len(heights)):
    print(weights[i]/(heights[i]**2))
```

Output:

```
22.77318640955005
25.351541373715524
25.30864197530864
```

- A more **pythonic** way is `zip`: it pairs up the elements of the two lists, and gives them as tuples, which the loop unpacks.

```
for w, h in zip(weights, heights):
    print(w/h**2)
```

Output:

```
22.77318640955005
25.351541373715524
25.30864197530864
```

Common mistakes

- **Trying to change a tuple:** `t[0] = 5` gives `TypeError: 'tuple' object does not support item assignment`. Use a list if the values have to change.
- **A tuple with one element:** `(5)` is just the number 5 in brackets. Add a comma to make a tuple: `(5,)` or `5,`.
- **Wrong number of variables when unpacking:** `x, y = (1, 2, 3)` gives `ValueError: too many values to unpack (expected 2)`.
- **Lists of different lengths in `zip`:** `zip` stops at the end of the shorter list, without an error message, so some elements may be left out.

Summary

Term / code	Meaning
`(1, 2, 3)`	create a tuple; `t[0]` and `t[1:3]` work as for a list
mutable / immutable	can / cannot be changed (list / tuple)
tuple as a key	allowed, e.g. `D[(1, 2)]`; a list is not
flat / nested	one dictionary with tuple keys / a dictionary of dictionaries
`t = 1, 2, 3`	implicit tuple, no `()` needed
`x, y = (10, 20)`	unpacking: assign the elements to several variables
`return a, b`	return multiple values (as a tuple)
`a, b = b, a`	swap the values of two variables
`d.items()`	the `(key, value)` tuples of a dictionary
`zip(L1, L2)`	pair up the elements of two lists
`for x, y in zip(L1, L2):`	loop over two lists together